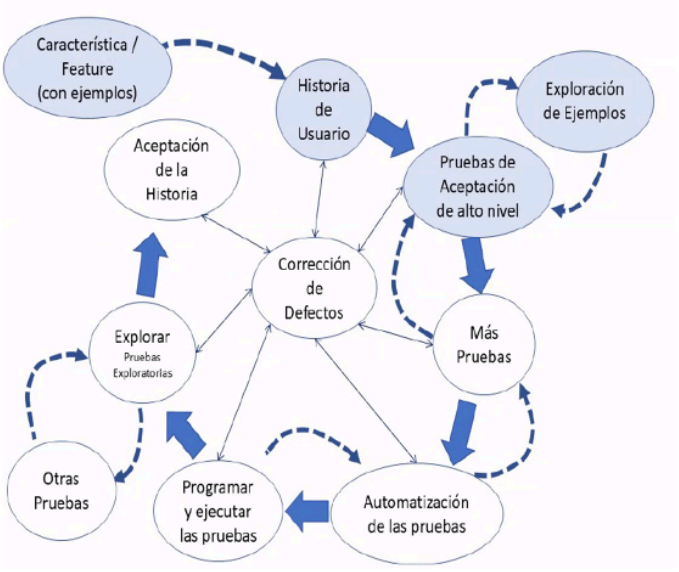
09/10 - Testing en Ambientes ágiles

1. Introducción al Testing Ágil

El testing en metodologías ágiles deja de ser una fase de cierre aislada para convertirse en una **actividad continua** que acompaña todo el ciclo de vida del desarrollo de software.

- Las pruebas se ejecutan en todas las etapas del desarrollo.
- Las pruebas pueden llegar a guiar el desarrollo, como ocurre en **TDD** (*Test-Driven Development*).



2. Agenda de Temas Clave

Los aspectos fundamentales que abarca el testing ágil son:

- Valores, principios y cultura del manifiesto ágil.
- Prácticas y roles/competencias del tester.
- **Cuadrantes del testing**, **Pirámide del testing** y **Automatización** (considerados los temas más relevantes).
- Desafíos culturales (con menor peso analítico en esta etapa).

3. Manifiesto del Testing Ágil

Este manifiesto redefine el enfoque tradicional de las pruebas comparándolo con la mentalidad ágil:

Enfoque Tradicional	Enfoque Ágil
Pruebas al final (fase de cierre estricta)	**Pruebas durante** todo el proceso (fase continua)
Encontrar defectos	**Prevenir defectos** proactivamente
Verificar funcionalidad (ejecutar sin errores)	**Entender lo que se prueba** (comprender negocio, funcionalidad y necesidades del cliente)
Romper el sistema	**Construir un mejor sistema** (enfoque constructivo)
El tester es el responsable de la calidad	**Responsabilidad de todo el equipo** por la calidad

Los Pilares del Nuevo Rol

- **El testing no es una fase**, es una actividad transversal.
- **Sé parte del equipo** (*Sé el tester* integrado).
- **El objetivo es crear**, no solo romper.
- **Calidad compartida**, todos somos responsables del sistema.

4. Comparativa: Ciclo de Vida Ágil vs. Tradicional

- **Ágil (Iterativo / Incremental):**
  - El testing representa la **entrega de valor continuo** bajo un esquema dinámico.
  - Sucede a lo largo de toda la iteración, sin esperar a finalizar el código.
- **Tradicional (Cascada):**
  - Esquema estático y secuencial que busca la conformidad estricta con una especificación previa.
  - El testing ocurre exclusivamente después del desarrollo e implementación.

5. Principios del Testing Ágil

Son guías específicas sobre la dirección del testing sin dictar un procedimiento rígido:

1. Se mueve hacia adelante en el proyecto de forma continua.
2. El testing no es una fase aislada.
3. Todo el equipo hace testing.
4. Reducir la latencia del feedback.
5. Las pruebas representan expectativas claras.
6. Mantener el código limpio y corregir defectos rápidamente.
7. Reducir la sobrecarga de documentación innecesaria.
8. Las pruebas forman parte de la definición de hecho (*Definition of Done*).
9. Pasar de probar al final a ser **Conducido por Pruebas**.

6. Prácticas Concretas

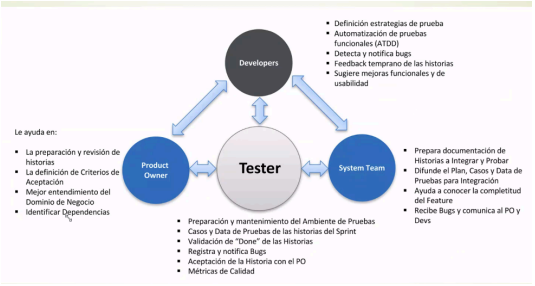
Técnicas que ayudan a llevar a la práctica el testing ágil:

1. Pruebas de unidad e integración automatizadas.
2. **TDD** (*Test-Driven Development*).
3. Pruebas de regresión a nivel de sistema automatizadas.
4. **ATDD** (*Acceptance Test-Driven Development* / Desarrollo conducido por pruebas de aceptación).
5. Pruebas exploratorias.
6. Control de versiones de las pruebas junto con el código.

7. Rol del Tester en Equipos Ágiles

En los equipos ágiles, el tester ocupa un **rol central** y colabora estrechamente con:

- El **PO** (*Product Owner*).
- El equipo de integración.
- Los **Developers** (desarrolladores).



8. Cuadrantes del Testing

Para entender los cuadrantes, primero hay que comprender los dos ejes principales:

- **Eje Vertical:** Divide las pruebas en **orientadas al negocio** (arriba, centradas en el usuario y requerimientos) y **orientadas a la tecnología** (abajo, centradas en el código y la arquitectura).
- **Eje Horizontal:** Divide las pruebas entre las que **guían el desarrollo** (izquierda, previenen defectos antes/durante la construcción) y las que **critican el producto** (derecha, evalúan el software ya construido para encontrar mejoras o fallas).

1. Cuadrante 1: Pruebas unitarias y de componentes (Inferior Izquierdo)

- **Orientación:** Tecnología + Guían el desarrollo.
- **Enfoque:** Son pruebas técnicas automatizadas que sirven como red de seguridad para los desarrolladores mientras escriben el código.
- **Propósito:** Verificar que las partes más pequeñas del software funcionen correctamente de forma aislada.
- **Ejemplos:** Pruebas unitarias y pruebas de componentes (a nivel de código).

2. Cuadrante 2: Pruebas funcionales y de aceptación (Superior Izquierdo)

- **Orientación:** Negocio + Guían el desarrollo.
- **Enfoque:** Definen los criterios de aceptación desde la perspectiva del usuario antes y durante la codificación. Suelen ser automatizables o guiadas por prototipos/ejemplos.
- **Propósito:** Asegurar que el sistema construya exactamente lo que el negocio y el usuario necesitan.

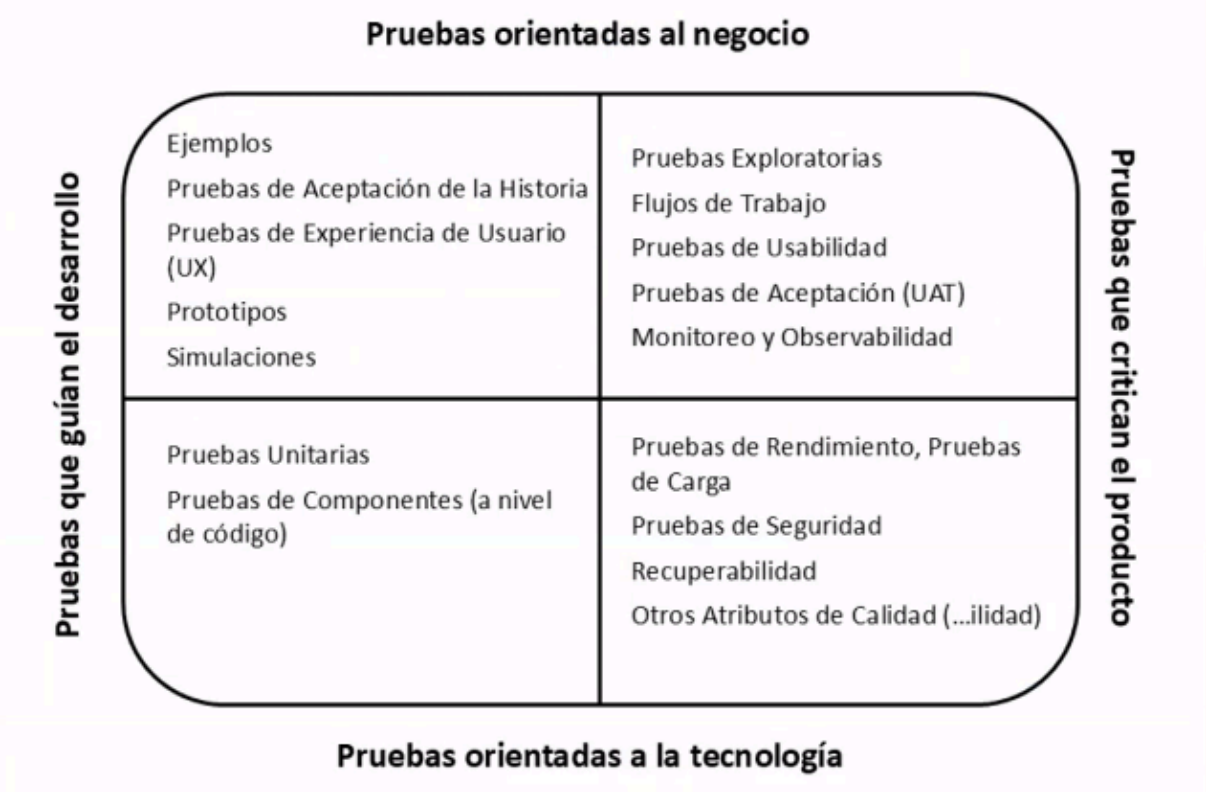
- **Ejemplos:** Ejemplos prácticos, pruebas de aceptación de historias de usuario, pruebas de experiencia de usuario (UX), prototipos y simulaciones.

3. Cuadrante 3: Pruebas exploratorias y de uso (Superior Derecho)

- **Orientación:** Negocio + Critican el producto.
- **Enfoque:** Pruebas principalmente manuales basadas en la intuición, experiencia y comportamiento real del usuario sobre el producto terminado o en etapas de integración.
- **Propósito:** Evaluar la usabilidad y descubrir defectos complejos o flujos no contemplados.
- **Ejemplos:** Pruebas exploratorias, validación de flujos de trabajo, pruebas de usabilidad, pruebas de aceptación de usuario (UAT), monitoreo y observabilidad.

4. Cuadrante 4: Pruebas no funcionales y rendimiento (Inferior Derecho)

- **Orientación:** Tecnología + Critican el producto.
- **Enfoque:** Pruebas técnicas respaldadas por herramientas especializadas de simulación y análisis.
- **Propósito:** Verificar los atributos de calidad del sistema (requerimientos no funcionales) bajo condiciones extremas o de producción.
- **Ejemplos:** Pruebas de rendimiento y carga, pruebas de seguridad, pruebas de recuperabilidad y otros atributos de calidad (...ilidad).



9. Pirámide del Testing Ágil vs. Tradicional

- **Pirámide Ágil (Invertida o adaptada al enfoque moderno):**
  - Se prioriza que las **pruebas unitarias y de componentes** acaparen el mayor tiempo y esfuerzo.
  - Los **tests de aceptación** ocupan entre un **5% y 15%** del esfuerzo total.
  - Los **tests de GUI** (interfaz gráfica) representan un **1% a 5%**, buscando evitar bugs visuales complejos al final.
  - *Resultado:* El proceso se acelera drásticamente gracias a la automatización eficiente.
- **Pirámide Tradicional:**
  - El **80% al 90%** de las pruebas son manuales, lo que vuelve el proceso lento y pesado.

10. Beneficios de la Automatización del Testing

- Rápida ejecución.
- Reusabilidad de los scripts de prueba.

- Mayor cobertura de código y escenarios.
- Mayor precisión y consistencia.
- Mayor alcance en las pruebas de regresión.
- Reducción de costos a mediano y largo plazo.